



SDD

System Design Document

Tempest Double

Riferimento	TD_SDD_ver.1.0
Versione	1.0
Data	16/01/2025
Destinatario	Prof. Carmine Gravino
Presentato da	NC21



Revision History

Data	Versione	Descrizione	Autori
10/12/2024	0.1	Prima stesura	AM
22/12/2024	0.2	Aggiunta dei Design Goal	MS
28/12/2024	0.3	Aggiunta delle sezioni 3.2, 3.3, 3.4	MS, CDB, AM
04/01/2025	0.4	Aggiunta capitoli 3.7, 4	MS, CDB, AM
06/01/2025	0.5	Revisione e correzione di eventuali errori	MS, CDB, AM
12/01/2025	0.6	Incorporato ODD	MS
16/01/2025	1.0	Revisione Finale e correzione degli errori	AM



Team Members

Nome	Ruolo nel Progetto	Acronimo	Informazioni di Contatto
Maksym Savitskyi	Team Member	MS	m.savitskyi@studenti.unisa.it
Chloe Dominique Briones	Team Member	CDB	c.briones@studenti.unisa.it
Alessandro Massimo	Team Member	AM	a.massimo1@studenti.unisa.it



Sommario

Revision History.....	2
Team Members.....	3
1. Introduzione	5
1.1 Scopo del Sistema	5
1.2 Obiettivi di Design (Design Goals)	5
1.3 Definizioni, Acronimi e Abbreviazioni	8
1.4 Riferimenti	9
1.5 Organizzazione del Documento	9
2. Architettura del Sistema Corrente	10
3. Architettura del Sistema Proposto	10
3.1 Panoramica sulla sezione.....	10
3.2 Decomposizione in Sottosistemi.....	10
3.3 Mapping Hardware/Software.....	12
3.4 Gestione dei Dati Persistenti	14
3.5 Controllo degli Accessi e Sicurezza	19
3.6 Controllo Globale del Software	20
3.7 Condizioni Limite	20
4. Servizi dei Sottosistemi.....	25
Glossario	28



1. Introduzione

1.1 Scopo del Sistema

La piattaforma è progettata per fornire un ambiente completo e intuitivo per la simulazione, il monitoraggio e l'ottimizzazione dei sistemi energetici. Si rivolge a un ampio spettro di utenti, tra cui professionisti del settore energetico, ricercatori, educatori e aziende, offrendo loro strumenti per comprendere, analizzare e ottimizzare il consumo e la distribuzione di energia.

Il sistema permette agli utenti di simulare scenari di consumo e produzione energetica in tempo reale, monitorare l'efficienza dei sistemi, identificare eventuali inefficienze e ottenere suggerimenti basati su analisi predittive. Inoltre, consente la creazione di modelli personalizzati che rappresentano configurazioni di sistemi energetici reali, come impianti residenziali, commerciali o industriali.

Tra i principali obiettivi ci sono:

- **Supportare la transizione energetica**, aiutando a integrare tecnologie come le energie rinnovabili e i sistemi di stoccaggio energetico.
- **Promuovere la sostenibilità**, fornendo strumenti per calcolare e ridurre l'impatto ambientale attraverso strategie di ottimizzazione.
- **Facilitare la formazione e la ricerca**, offrendo un ambiente interattivo e didattico per la simulazione e l'analisi di scenari complessi.

La piattaforma garantisce accessibilità da qualsiasi dispositivo connesso a Internet, eliminando la necessità di installazioni locali e rendendola ideale per un utilizzo flessibile sia individuale che collaborativo.

1.2 Obiettivi di Design (Design Goals)

Nella presente sezione si andranno a presentare i Design Goals, ovvero le qualità sulle quali il sistema deve essere focalizzato, formalizzati esplicitamente così che qualsiasi importante decisione di design può essere fatta consistentemente seguendo lo stesso insieme di design goal.

Seguendo le linee guida del libro Bernd Bruegge – Object Oriented Software Engineering i design goal sono stati suddivisi nelle seguenti categorie:

- **Performance**: includono i requisiti di spazio e velocità imposti sul sistema.
- **Dependability**: determinano quanto sforzo deve essere speso per minimizzare i fallimenti del sistema (crash, falle di sicurezza) e le loro conseguenze.



- **Maintenance:** determina quanto sforzo è necessario per modificare il sistema dopo il suo rilascio.
- **End User:** includono qualità che sono desiderabili dal punto di vista dell'utente, ma che non sono state coperte dai criteri di Performance e Dependability.

Ciascun design goal è descritto da:

- **Rank:** specifica un valore di priorità compreso tra 1 e 12 (1 massima e 12 minima).
- **ID Design Goal:** un identificatore univoco e un nome esplicativo.
- **Descrizione:** una descrizione del design goal.
- **Categoria:** ovvero la categoria di appartenenza del design goal.
- **RNF di origine:** ovvero il requisito non funzionale che lo ha generato.

Design goals

Rank	ID Design Goal	Descrizione	Categoria	RNF di origine
2	DG_1 Tempi di risposta	Il sistema deve garantire un tempo di risposta non superiore a 3 secondi.	Performance	RNF_P_5
6	DG_2 Quantità di dati	Il sistema deve permettere di gestire molti scenari, asset, record storici e dati degli ambienti, che per loro natura richiedono molto spazio.	Performance	RNF_P_3
1	DG_3 Affidabilità delle operazioni	Il sistema deve garantire che tutte le operazioni avvengano con successo.	Dependability	RNF_A_1
11	DG_4 Fallimento di sistema	Il sistema deve sapersi comportare in situazioni di fallimento notificando l'utente, tramite appositi messaggi.	Dependability	RNF_A_3



7	DG_5 Precisione	Considerando gli ambiti di utilizzo del sistema proposto, la precisione delle simulazioni e dei dati deve essere massima.	Dependability	RNF_A_6
10	DG_6 Sicurezza dei dati	Il sistema deve garantire la massima sicurezza dei dati conservati, utilizzando protocolli di comunicazione sicuri, conservando in maniera sicura i dati persistenti.	Dependability	RNF_A_2
5	DG_7 Manutenibilità	Il sistema deve essere facilmente manutenibile ed estendibile.	Maintenance	RNF_S_1
4	DG_8 Estensibilità	Il sistema si presta facilmente all'aggiunta di nuove funzionalità, come ad esempio nuovi tipi di asset (per esempio quelli non esistenti al momento della pubblicazione di questa versione del sistema).	Maintenance	RNF_S_2
3	DG_9 Facilità d'Uso	Il sistema deve risultare facilmente comprensibile ed utilizzabile anche da un'utenza meno esperta.	End User	RNF_U_4
8	DG_10 Interfaccia intuitiva	L'interfaccia utente della piattaforma deve permettere di eseguire azioni in modo chiaro e semplice, rendendo ben esplicita la funzionalità di ogni elemento visuale.	End User	RNF_U_2



9	DG_11 Feedback esplicito	Ogni azione all'interno della piattaforma in seguito ad un'interazione dell'utente deve comunicare un chiaro feedback allo stesso.	End User	RNF_U_3
12	DG_12 Deployment Semplificato	Il sistema deve essere facilmente deployabile anche su una singola macchina, esponendo l'applicazione web sulla rete e accedendo a tutti i servizi esterni necessari.	End User	RNF_PA_1

Trade-Off

Trade-Off	Descrizione
Sistema multiutente vs complessità	La possibilità del sistema di essere utilizzato da più utenti contemporaneamente comporta un elevato aumento di complessità del sistema, con conseguente aumento di costi e tempo di sviluppo e riducendo la portabilità del sistema.
Tempi di risposta vs sicurezza	Il livello di sicurezza implementato è relativamente basso, dando priorità alla velocità delle operazioni prima della sicurezza dei dati.

1.3 Definizioni, Acronimi e Abbreviazioni

Di seguito una lista di definizioni, acronimi e abbreviazioni:

- **AD:** Activity Diagram;
- **COTS:** Commercial Off The Shelf;
- **CRUD:** Create, Read, Update, Delete;
- **DBMS:** Database Management System;
- **GUI:** Graphic User Interface;
- **JPA:** Java Persistence API;
- **NP:** Navigational Path;
- **ODD:** Object Design Document;



- **RAD:** Requirements Analysis Document;
- **RF:** Requisito Funzionale;
- **RNF:** Requisito Non Funzionale;
- **SD:** Sequence Diagram;
- **SDD:** System Design Document;
- **UML:** Unified Modeling Language;
- **WBS:** Work Breakdown Structure.

1.4 Riferimenti

Di seguito una lista di riferimenti ad altri documenti utili durante la lettura:

- Statement Of Work;
- System Design Document;
- Test Plan;
- Test Case Specification.

1.5 Organizzazione del Documento

Il presente documento di System Design è strutturato nel seguente modo:

1. **Introduzione:** la seguente sezione contiene lo scopo del prodotto, i Design Goals e una panoramica sulle definizioni, acronimi e abbreviazioni presenti nel documento.
2. **Architettura del Sistema Corrente:** viene offerta una descrizione sullo stato attuale del sistema già presente.
3. **Architettura del Sistema Proposto:** viene descritto come il sistema sarà definito e partizionato in sottosistemi, il loro mapping Hardware/Software, la gestione dei dati persistenti. Verranno poi presentate la struttura dei singoli sottosistemi e le boundary conditions riguardanti l'intero sistema.
4. **Servizi dei Sottosistemi:** in questa sezione vengono descritti i servizi dei sottosistemi precedentemente identificati.
5. **Glossario:** descrive i termini tecnici presenti nel documento.



2. Architettura del Sistema Corrente

Al momento, non esiste alcun software che condensi l'interezza delle funzionalità di Tempest Double in un unico servizio. Il mercato delle possibili alternative a questo software è pertanto incredibilmente frammentato e non esiste una reale architettura a cui è possibile confrontare in maniera ragionevole il sistema.

3. Architettura del Sistema Proposto

3.1 Panoramica sulla sezione

Il sistema proposto è basato sullo stile architetturale Three Tier, implementato utilizzando Spring MVC. Il motivo della presente scelta è che tale architettura è perfetta per lo sviluppo di web application come il nostro sistema, poiché la separazione della logica di presentazione da quella di elaborazione, migliora una serie di qualità, tra le quali:

- Leggibilità
- Manutenzione
- Riutilizzo

Nello sviluppo del sistema verranno usati HTML5, CSS3, JavaScript e Thymleaf per la parte di front-end e la generazione delle view.

Per la logica applicativa e quindi il back-end sarà utilizzato Java SPRING.

Per la gestione del database saranno usati:

- **Spring JPA** per il collegamento al database;
- un database open source **MySQL**, che può essere hostato localmente oppure usando servizi di Amazon.

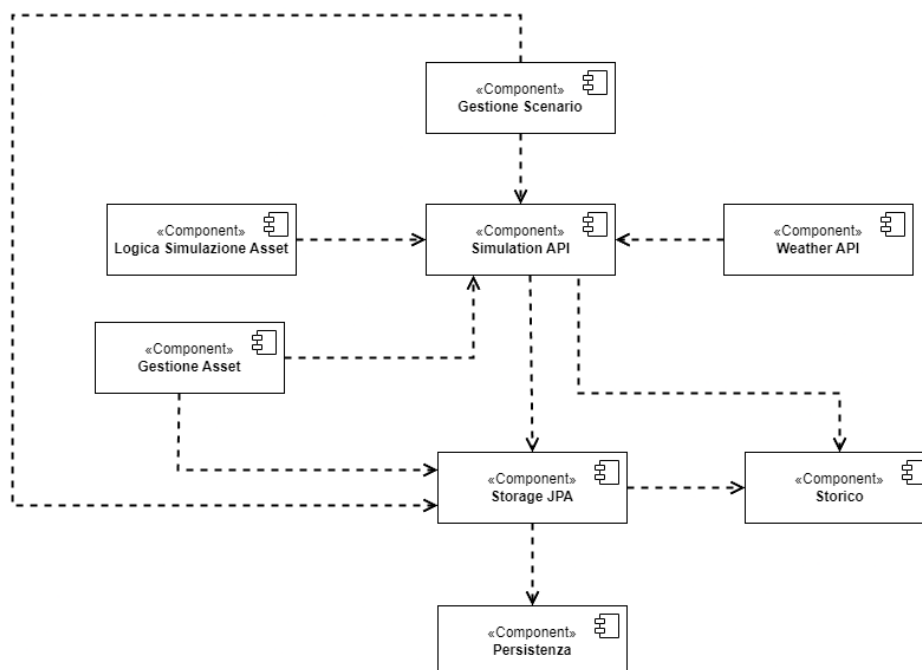
3.2 Decomposizione in Sottosistemi

I sottosistemi individuati sono:

- **Simulation API:** si occupa della gestione della simulazione (azioni di avvio/sospensione) e generazione dei dati intermedi della simulazione;
- **Logica Simulazione Asset:** Responsabile delle logiche della simulazione vera e propria del comportamento di ogni singolo asset;
- **Gestione Scenario:** Offre servizi CRUD sullo scenario e la sua visualizzazione interattiva grafica nella GUI;

- **Gestione Asset:** Si occupa della rappresentazione grafica degli asset e la loro configurabilità dalla parte dell'utente;
- **Storico:** Permette la visualizzazione delle simulazioni effettuate in passato e la visualizzazione di tutti i passi facenti parte della simulazione;
- **Persistenza:** Si occupa di gestire la persistenza dei dati con un database;
- **Storage JPA:** Si interpone tra i vari sottosistemi e il sottosistema di Persistenza.

Sono mostrate di seguito le dipendenze tra i diversi sottosistemi attraverso un Component Diagram UML.



Alcuni sottosistemi saranno gestiti da componenti COTS (Commercial Off The Shelf), di seguito un elenco:

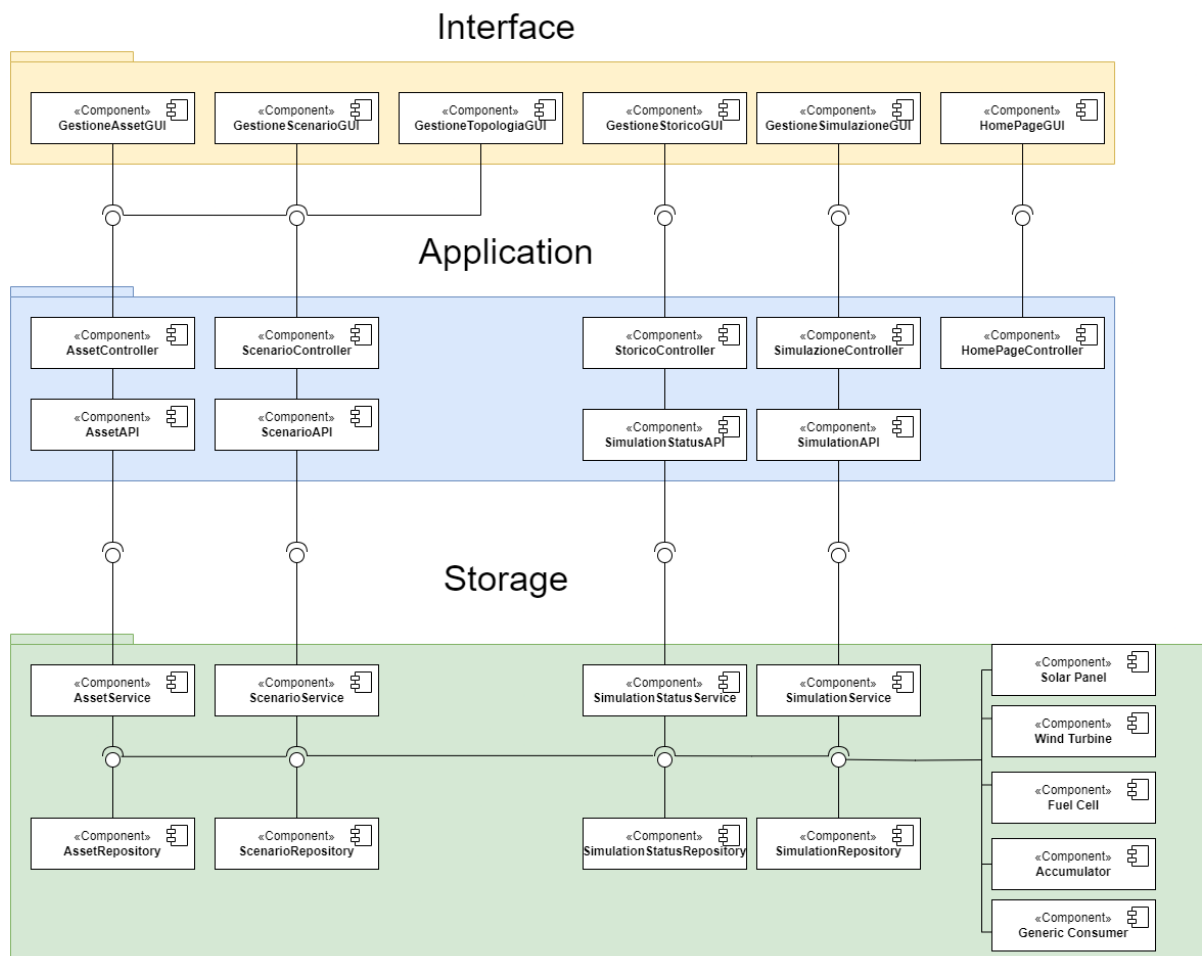
- **Storage JPA** verrà gestito da Spring Data JPA;
- **Persistenza** sarà gestita attraverso un DBMS relazionale MySQL.

Di seguito una vista dettagliata di ciascun sottosistema evidenziando le componenti principale:

- **GUI:** Graphic User Interface, che contiene le varie view che saranno renderizzate per creare le pagine web da mostrare all'utente;
- **Controller:** si occupa della logica per il controllo del sistema;

- **Service:** si occupa della logica di business;
- **Repository:** si occupa di fornire accesso ai dati persistenti.

Diagramma Architeturale

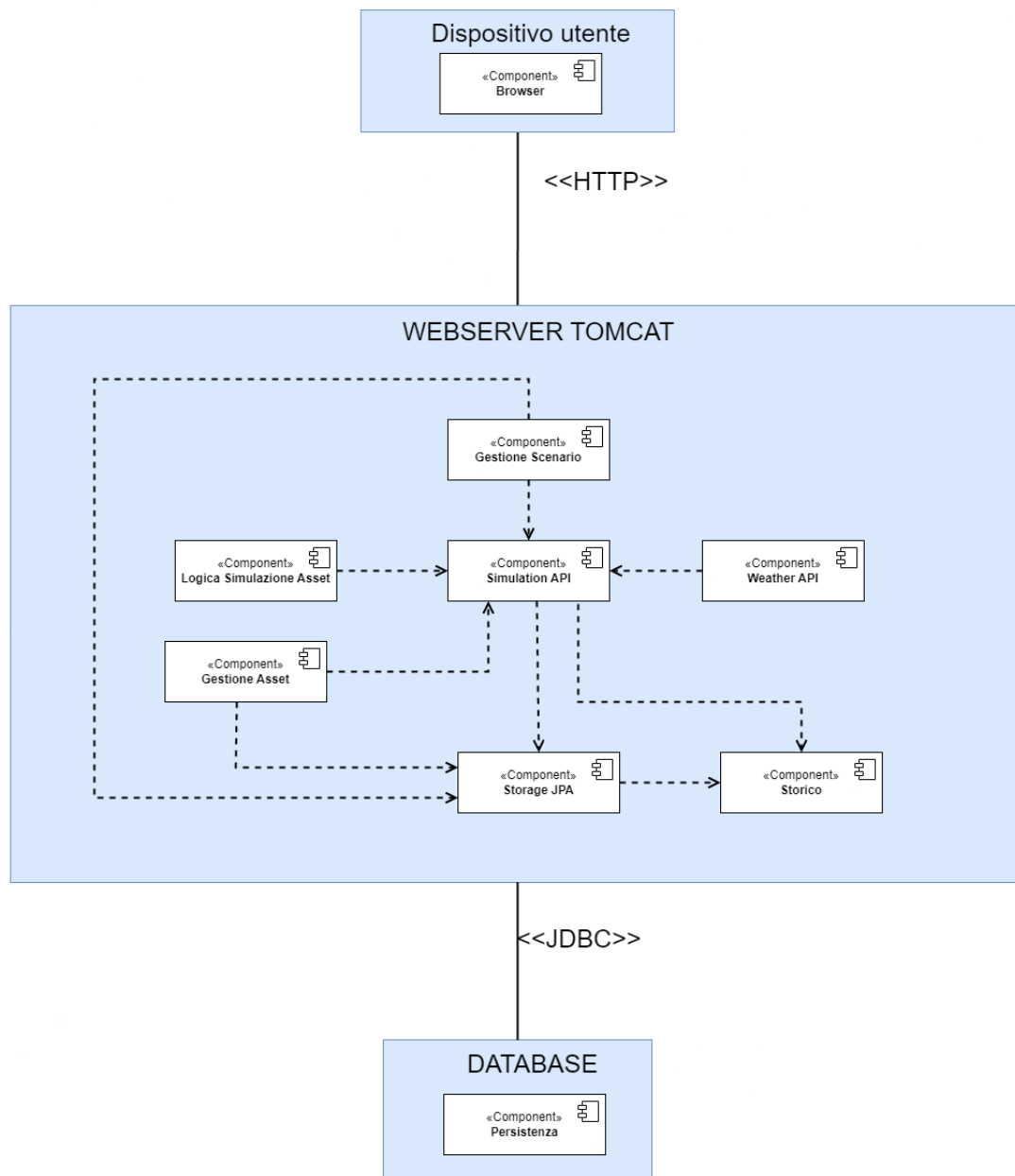


3.3 Mapping Hardware/Software

L'applicazione web che verrà sviluppata si basa su una piattaforma hardware costituita da un server, il quale gestisce le richieste effettuate dai client attraverso qualsiasi dispositivo dotato di browser e connessione a Internet. Poiché il sistema è una web application che risiede su un web server e si basa su un'architettura non distribuita, tutte le sue componenti sono ospitate su un unico nodo.



Di seguito un UML Deployment Diagram che descrive il mapping hardware/software:



3.4 Gestione dei Dati Persistenti

Introduzione

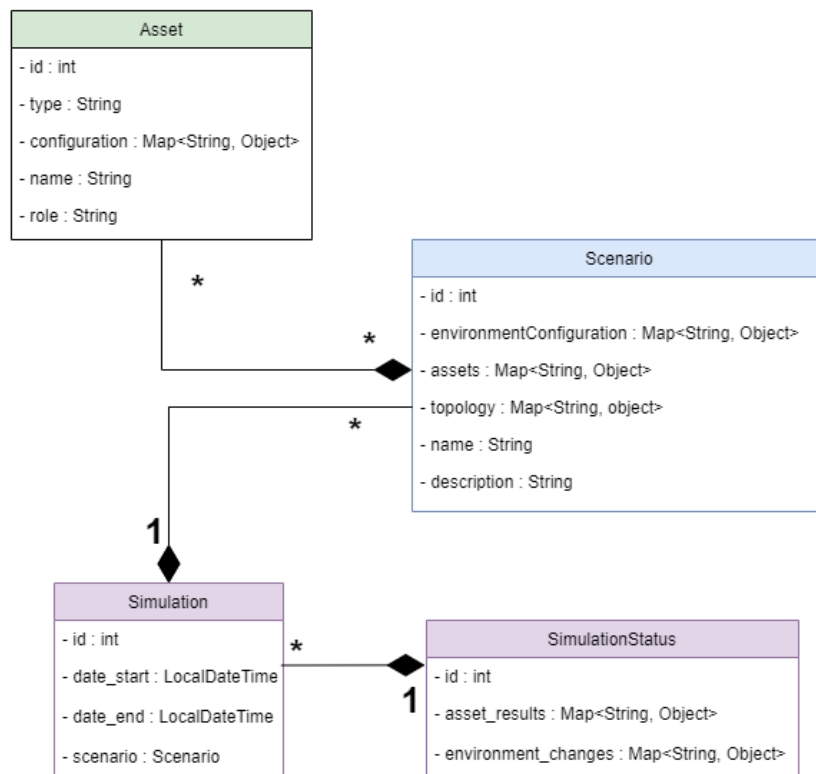


Per la gestione del salvataggio dei dati persistenti del sistema si è deciso di utilizzare un database relazionale, al fine di gestire agevolmente l'accesso concorrente ai dati e contemporaneamente garantire la consistenza dei dati tramite l'utilizzo di un DBMS.

La scelta di utilizzo di un DBMS è stata presa al fine di mantenerci quanto più possibile coerenti con i design goals stabiliti, potendo contare su:

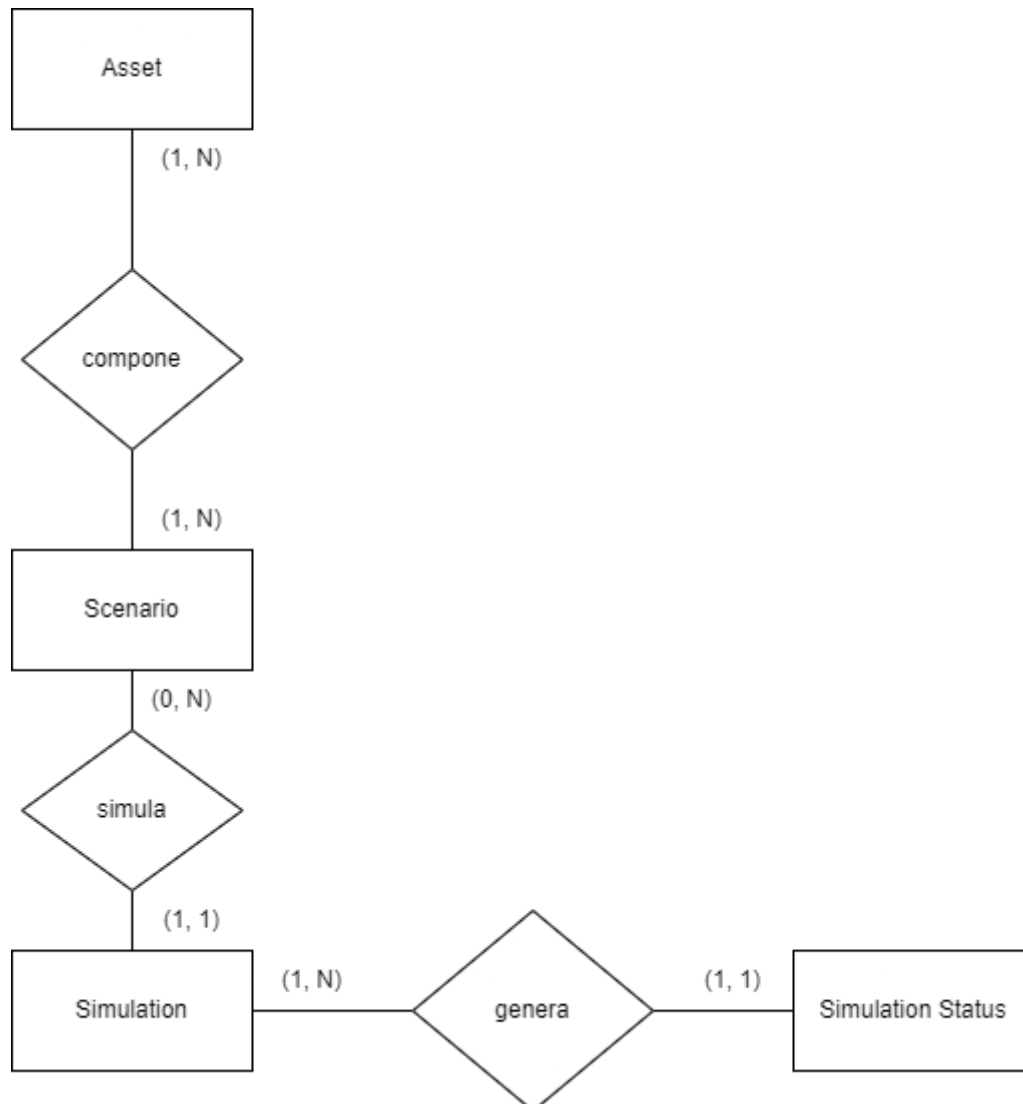
- **Imposizioni di vincoli di integrità sui dati:** poiché un DBMS permette di specificare diversi tipi di vincoli per mantenere l'integrità dei dati e controlla che tali vincoli siano soddisfatti quando la base di dati cambia;
- **Privatezza dei dati:** garantita dal fatto che un DBMS permette un accesso protetto ai dati. Utenti diversi possono avere accesso a diverse porzioni della base di dati e possono essere abilitati a diverse operazioni su di esse;
- **Affidabilità dei dati:** un DBMS offre dei metodi per salvare copie dei dati e per ripristinare lo stato della base di dati in caso di guasti software e hardware;
- **Atomicità delle operazioni:** data dal fatto che un DBMS permette di effettuare sequenze di operazioni in modo atomico. Ciò significa che l'intera sequenza di operazioni viene eseguita con successo oppure nessuna di queste operazioni ha alcun effetto sui dati della base. L'atomicità delle transazioni permette di mantenere uno stato della base di dati consistente con la realtà modellata.

CD_SDD: Entity Class Diagram



ER: Schema ER del database

SDD Tempest Double



Dizionario dei Dati

Di seguito si mostrano gli attributi per ogni entità individuata.

Nome Entità	Asset		
Descrizione	Rappresenta una risorsa che può essere utilizzata nei diversi scenari.		
Nome campo	Tipo	Vincolo di chiave	Altri vincoli
id	Int (32)	PRIMARY KEY	NOT NULL AUTO - INCREMENT



type	Varchar (10)		NOT NULL
configuration	JSON		NOT NULL
name	Varchar (20)		NOT NULL UNIQUE
role	Varchar (10)		NOT NULL

Nome Entità	Scenario		
Descrizione	Definisce un contesto o un insieme di condizioni in cui gli asset vengono utilizzati. Gli scenari possono rappresentare strategie o ipotesi per simulazioni.		
Nome campo	Tipo	Vincolo di chiave	Altri vincoli
id	Int (32)	PRIMARY KEY	NOT NULL AUTO - INCREMENT
environment_configuration	JSON		NOT NULL
assets	JSON		NOT NULL
scenario_name	Varchar (20)		NOT NULL
scenario_description	TEXT		
topology	JSON		NOT NULL



Nome Entità	Simulation		
Descrizione	Rappresenta un'istanza di esecuzione basata su uno scenario. Una simulazione è un modello operativo utilizzato per analizzare o prevedere risultati.		
Nome campo	Tipo	Vincolo di chiave	Altri vincoli
id	Int (32)	PRIMARY KEY	NOT NULL AUTO - INCREMENT
date_start	Datetime		NOT NULL
date_end	Datetime		
scenario_id	Int (32)	FOREIGN KEY	NOT NULL

Nome Entità	Simulation Status		
Descrizione	Rappresenta un'istanza di esecuzione basata su uno scenario. Una simulazione è un modello operativo utilizzato per analizzare o prevedere risultati.		
Nome campo	Tipo	Vincolo di chiave	Altri vincoli
id	Int(32)	PRIMARY KEY	NOT NULL AUTO - INCREMENT
assets_results	JSON		NOT NULL
environment_changes	JSON		NOT NULL

3.5 Controllo degli Accessi e Sicurezza

Poiché l'architettura dell'applicazione è di tipo monoutente, non è prevista alcuna gestione dei permessi.



3.6 Controllo Globale del Software

Il sistema Tempest Double è un sistema interattivo per cui ogni funzionalità viene avviata in seguito ad un comando impartito dall'utente tramite l'uso di un'interfaccia grafica. Quindi quando un utente vuole accedere ed utilizzare una funzionalità del sistema può farlo tramite l'interfaccia grafica la quale selezionerà il controllo corrispondente. L'azione scatenerà un evento, il quale verrà gestito dal suo handler. L'handler a sua volta indirizzerà il controllo del flusso di eventi al sottosistema che si occupa della logica di controllo e gestore del controllo che poi si rivolge ai servizi per la logica applicativa.

Per tali motivi il sistema utilizzerà un meccanismo di controllo del flusso di tipo event-driven essendo una web-application.

3.7 Condizioni Limite

Nel presente paragrafo verranno presentate le boundary conditions inerenti all'avvio del sistema, spegnimento del sistema, fallimento del sistema ed errore di accesso ai dati persistenti. Sono descritti solo scenari che riguardano l'installazione locale del sistema.

Avvio del sistema

Identificativo	UCBC_1 - Avvio del sistema	Data	04/01/2025
		Versione	1.0
		Autori	Maksym Savitskyi
Descrizione	L'UC permette l'avvio del sistema		
Attore Principale	Utente		
Attori Secondari	NA		
Entry Condition	L'utente accede al sistema		
Entry Condition <i>On success</i>	Il sistema viene avviato correttamente		
Entry Condition	Il sistema non viene avviato		



<i>On failure</i>		
Flusso di eventi principale		
1	Utente	Avvia il sistema cliccando icona sul desktop
2	Sistema	Verifica la disponibilità del servizio database e, se disponibile, rende disponibili i suoi servizi e rende le sue funzionalità disponibili all'utente.

Identificativo	UCBC_2 - Spegnimento del sistema	Data	04/01/2025
		Versione	1.0
		Autori	Maksym Savitskyi
Descrizione	L'UC permette lo spegnimento del sistema		
Attore Principale	Utente		
Attori Secondari	NA		
Entry Condition	L'Utente accede al sistema AND Il Sistema è stato precedentemente avviato AND Il Sistema non è stato ancora spento		
Entry Condition <i>On success</i>	Il sistema viene spento correttamente		
Entry Condition <i>On failure</i>	Il sistema non viene spento		
Flusso di eventi principale			
1	Utente	Invia un segnale di spegnimento al sistema	



2	Sistema	Controlla che non ci siano simulazioni ancora in corso e, se non ci sono, termina l'esecuzione del sistema.
I Flusso di eventi alternativo: Ci sono ancora simulazioni in corso		
2.a1	Sistema	Notifica l'utente che ci sono ancora simulazioni in corso.
2.a2	Sistema	Ferma correttamente le simulazioni in corso, salvando i dati necessari.
2.a3	Sistema	Chiude la connessione in database e termina il sistema.
2.a4	Sistema	Notifica l'utente dell'avvenuto spegnimento del sistema.

Fallimento Sistema

Identificativo	UCBC_3 - Fallimento del sistema	Data	04/01/2025
		Versione	1.0
		Autori	Maksym Savitskyi
Descrizione	L'UC definisce il comportamento del Sistema in caso di fallimento.		
Attore Principale	Utente		
Attori Secondari	NA		
Entry Condition	Il Sistema viene terminato inaspettatamente		



Entry Condition <i>On success</i>		Il Sistema viene riavviato correttamente
Entry Condition <i>On failure</i>		Il Sistema non viene riavviato
Flusso di eventi principale		
1	Utente	Include UCBC_1

Errore di Accesso ai Dati Persistenti

Identificativo	UCBC_4 - Errore di Accesso ai Dati Persistenti	Data	04/01/2025
		Versione	1.0
		Autori	Maksym Savitskyi
Descrizione	L'UC descrive il comportamento del sistema qualora fosse impossibile accedere ai dati persistenti o questi risultassero corrotti.		
Attore Principale	Utente		
Attori Secondari	NA		
Entry Condition	Il Sistema non può accedere ai dati persistenti OR I dati persistenti risultano corrotti		
Entry Condition On success	Il Sistema riprende il normale funzionamento		
Entry Condition On failure	Il Sistema non riprende il normale funzionamento		
Flusso di eventi principale			



1	Sistema	Notifica l'utente dell'impossibilità di accedere ai dati persistenti.
2	Sistema	Cessa di processare eventuali richieste ai servizi che necessitano di accedere ai dati persistenti e risponde a tutte le richieste con un messaggio di errore.
3	Utente	Include UCBC_2
4	Utente	Include UCBC_1

4. Servizi dei Sottosistemi

In questa sezione saranno descritti i servizi di ogni sottosistema precedentemente elencato.

Sottosistema Simulation API

Servizio	Descrizione	Interfaccia
Start Simulazione	Avvia una simulazione, in base al nome dello scenario.	SimulationService
Stop Simulazione	Ferma una simulazione, in base al nome dello scenario.	SimulationService

Sottosistema Logica Simulazione Asset

Servizio	Descrizione	Interfaccia
Simulazione di un asset consumatore generico.	Simula il comportamento di un asset consumatore in base ai dati sul consumo elettrico passati in input e agli attributi dell'oggetto simulato.	GenericConsumer
Simulazione di un accumulatore.	Simula il comportamento di un asset accumulatore in base ai dati sulla produzione elettrica passati in input e agli attributi dell'oggetto simulato.	Accumulator
Simulazione di un pannello solare	Simula il comportamento di un asset pannello solare in base ai dati ambientali passati in input e agli attributi dell'oggetto simulato.	SolarPanel
Simulazione di un Impianto Eolico	Simula il comportamento di un asset impianto eolico in base ai dati ambientali passati in input e agli attributi dell'oggetto simulato.	WindTurbine



Simulazione di una Fuel Cell	Simula il comportamento di un asset fuel cell in base ai dati ambientali passati in input e agli attributi dell'oggetto simulato.	FuelCell
------------------------------	---	----------

Sottosistema Gestione Scenario

Servizio	Descrizione	Interfaccia
Creazione Scenario	Permette di creare uno scenario che include gli asset inseriti dalla GUI e permette di dare un nome univoco che lo identifichi.	ScenarioService
Eliminazione Scenario	Permette di eliminare uno scenario in base al suo nome univoco.	ScenarioService
Eliminazione di più scenari	Permette di eliminare più scenari passando in input più identificativi degli scenari.	ScenarioService
Caricamento Scenario	Permette di recuperare uno scenario esistente in base al suo nome.	ScenarioService
Recupero di tutti gli Scenari	Permette di recuperare tutti gli scenari registrati nel sistema.	ScenarioService
Aggiornamento Scenario	Permette di aggiornare uno scenario esistente con nuovi dati.	ScenarioService

Sottosistema Gestione Asset

Servizio	Descrizione	Interfaccia
Creazione Asset	Permette di creare un nuovo asset specificando i suoi attributi.	AssetService



Eliminazione Asset	Permette di eliminare un asset specificando il suo id.	AssetService
Aggiornamento Asset	Permette di aggiornare un asset esistente con nuovi dati.	AssetService
Recupero Asset	Permette di recuperare un asset già registrato.	AssetService
Recupero di più Asset	Permette di recuperare più asset già registrati.	AssetService

Sottosistema Storico

Servizio	Descrizione	Interfaccia
Creazione record storico	Crea un record storico della simulazione in esecuzione.	SimulationStatusService
Eliminazione di un record storico	Elimina un record storico indicando il suo id.	SimulationStatusService
Eliminazione di più record storici	Elimina più record storici passando più id.	SimulationStatusService
Recupero di un record storico	Recupera un record storico in base al suo id.	SimulationStatusService
Recupero di più record storici	Recupera tutti i record storici che fanno parte della simulazione.	SimulationStatusService



Glossario

Sigla/Termine	Definizione
Activity Diagram	Diagramma UML che rappresenta il flusso di attività in un processo o sistema.
Application Programming Interface (API)	Insieme di regole che consente a componenti software diversi di comunicare tra loro.
Asset	Entità simulata dal sistema, divisa in consumatori (che consumano energia o risorse) e produttori (che generano energia o offrono servizi).
Database	Sistema organizzato per la gestione e l'archiviazione dei dati.
Framework	È un'architettura logica di supporto sulla quale un software può essere progettato e realizzato.
Internet of Things	Ecosistema di dispositivi interconnessi che comunicano tra loro tramite Internet.
Java EE	Una versione di Java utilizzata per lo sviluppo di applicazioni Enterprise.
Mock-up	Una rappresentazione visuale della User Interface, utile al committente per capire come il prodotto sarà fruibile all'utente finale.
Persistenza dei Dati	Proprietà che garantisce che le informazioni memorizzate dal sistema siano salvate in modo stabile e recuperabili anche in caso di interruzioni.
Piattaforma	Base software o hardware su cui sono sviluppate o eseguite applicazioni.



Scenari	Rappresentazioni informali di situazioni operative del sistema dal punto di vista dell'utente finale.
Simulazione	Processo che permette di riprodurre scenari virtuali per analizzare il comportamento di dispositivi IoT e reti di asset, al fine di ottimizzarne il funzionamento.
Spring JPA	Strumento del framework Spring che semplifica la gestione della persistenza dei dati utilizzando l'API Java Persistence.
Topologia	Struttura che rappresenta le connessioni tra diversi asset (produttori e consumatori) all'interno di una simulazione.



ODD

Object Design
Document

Tempest Double

Design Pattern

1. Il Service-Repository Pattern

È un design pattern ampiamente utilizzato nello sviluppo di applicazioni enterprise, in particolare con framework come Spring. Questo pattern si basa su una separazione chiara delle responsabilità tra diversi livelli dell'applicazione:

- **Entità:** rappresentano il modello di dominio e mappano i dati della base di dati.
- **Repository:** gestiscono l'accesso ai dati e contengono le operazioni CRUD e query specifiche.
- **Service:** contengono la logica di business, orchestrando le operazioni sui dati fornite dai repository.
- **Controller:** interagiscono con i livelli superiori (ad esempio, interfacce utente o API).

Il pattern consente modularità, riusabilità e testabilità, favorendo un'architettura pulita e scalabile. Inoltre, promuove l'applicazione dei principi SOLID, come il **Single Responsibility Principle** e il **Dependency Inversion Principle**, garantendo un'alta manutenibilità del codice.

2. MVC (Model-View-Controller)

MVC (Model-View-Controller) è un design pattern architetturale che separa un'applicazione in tre componenti principali:

- **Model:** Rappresenta il livello dati e la logica di business. Include oggetti che mappano i dati del dominio e gestisce l'accesso e la manipolazione di tali dati. Questo componente viene gestito dal design pattern descritto sopra.
- **View:** Si occupa della presentazione e dell'interfaccia utente. Si occupa quindi di mostrare i file statici, come ad esempio immagini o CSS.
- **Controller:** Funziona come mediatore tra il **Model** e la **View**, gestendo le richieste dell'utente, invocando i servizi necessari e restituendo le risposte.

Questo pattern permette di separare le responsabilità per migliorare la manutenibilità, la scalabilità e il riutilizzo del codice ed inoltre funziona particolarmente bene quando abbinato all'utilizzo del **Service-Repository Pattern**.

3. Converter Pattern

Il **Converter Pattern** è un design pattern utilizzato per convertire dati tra due rappresentazioni diverse, come oggetti di dominio e formati di trasmissione o archiviazione.

Viene implementato per gestire le conversioni tra formati diversi in modo centralizzato, riducendo duplicazione di codice, il testing e la complessità.

Nel caso di Tempest Double, la classe **JsonConverter** rappresenta un esempio del pattern, convertendo oggetti **Map<String, Object>** in stringhe JSON per la persistenza o il trasferimento e viceversa.

4. Dependency Injection (DI)

Dependency Injection (DI) è un principio architetturale che facilita la gestione delle dipendenze in un'applicazione, delegando la creazione e l'iniezione degli oggetti necessari a un framework (come Spring).

L'utilizzo di questo pattern riduce il coupling tra i componenti e migliora la testabilità e la modularità del codice.

Annotazioni come **@Autowired**, **@Service**, **@Repository** e **@Controller** sono utilizzi di DI. Ad esempio, un oggetto come **ScenarioRepository** viene iniettato automaticamente in **ScenarioServiceImpl** dal framework.